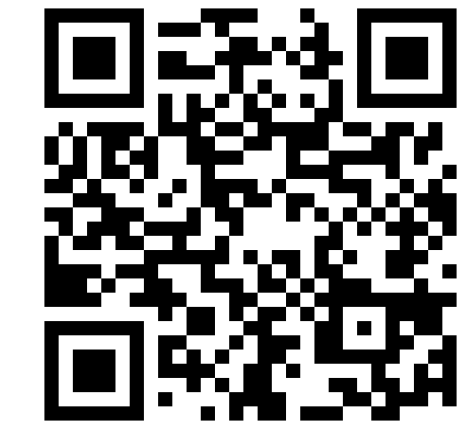


Agentic AI for Earth Science Software: a field report from building Worldscope

David Matthew Hall · NVIDIA · Berlin, Germany

An interactive Earth-system workbench. The AI agents in it are first-class users of every capability, with the same access a person has.



See it run
Source & walkthrough video

WHAT IT DOES

An **open-source Earth-2 tool** for weather, climate, and Earth-observation data, built to lower the barrier to entry for people who think visually. It runs on any device and uses whatever NVIDIA hardware is present for data services and AI inference. On a single desktop **DGX Spark** it runs the vast majority of AI weather models locally. It complements **Earth-2 Command Center**, which remains the right tool for photorealistic visualization and Omniverse digital twins.

DATA, FILES & FORMATS

- Reads **NetCDF**, **GRIB2**, **Zarr**, **GeoTIFF** across many grid types, including **HEALPix rendered natively** rather than interpolated to lat–lon first
- Opens remote files, and moves whole files or single fields between machines
- Many online sources: weather, climate, satellite observation
- Active weather caching, so animation plays at speed
- Every display carries units and a legend

MODELS & LIVE EARTH

- Launches **Earth2Studio** forecast, diagnostic, and assimilation runs from a GUI
- Ensemble member, vertical level, and time are all selectable axes
- Live conditions with animated winds and currents from several sources
- Live feeds: aircraft, ships, earthquakes, satellites
- Meteograms at a point, and side-by-side comparison through a swipe window

VIEWS & OUTPUT

- 2D and 3D at every scale, globe down to city blocks with buildings
- Sun, Moon, Mars, and other bodies for planetary missions
- Exports images, PDFs, and animations at chosen format and frame rate
- Multi-window, multi-tab shell modeled on VS Code, with a demo mode that hides the UI
- iPhone and iPad touch support in development

AGENTS AS USERS

- Every capability sits behind an **MCP server**. Agents call the same 450 commands the UI calls
- An agent loads data, runs a model, compares results, and drives the same scene the human sees
- **Several agents can work one scene at once**
- AI-driven demos: a slideshow of features that stays fully interactive

NEXT weather radar · Jupyter notebooks · large ensembles and forecast tracks · vertical-profile analysis · Earth-observation workflows such as change detection · AI model chains running global forecast → regional impact → decision support

4½ mo

PROOF OF CONCEPT
TO PLATFORM

1,444

COMMITTS
(100% AGENT-WRITTEN)

181k

LINES OF
SOURCE

450

AGENT-CALLABLE
COMMANDS

38

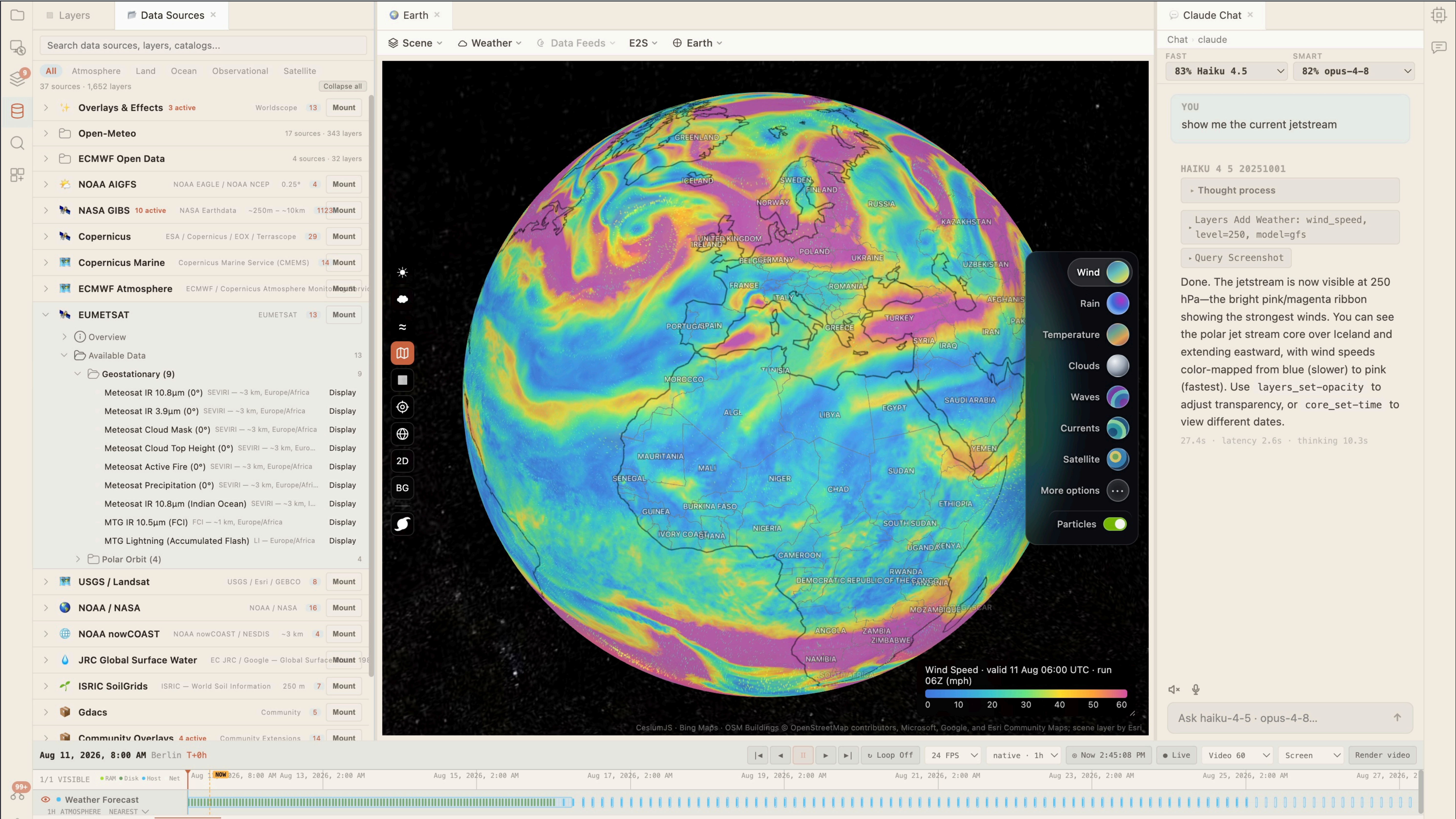
COMMAND
MODULES

577

TEST
FILES

2 + 1

AI ASSISTANTS +
ONE HUMAN



Worldscope. A global forecast field on the interactive globe, with the layer explorer at left and an agent session at right. Through the Model Context Protocol the agent drives the same scene the human sees, sharing its layers, its camera, and its time cursor.

What building it taught me

- 1

The bottleneck moved downstream.
For well-specified features, agents produced implementations roughly **a hundred times faster** than I could have written them. The cost of implementation collapsed. The effort moved into specification, integration, and verification, none of which got cheaper.
- 4

Tests caught regressions, not wrongness.
Agent-written tests found breakage, but **could not establish that a complex interactive workflow behaved correctly end to end**. Code that is plausible, runnable, and wrong is the dangerous case in scientific software, and a green test suite does not catch it.

- 2

Holding the whole thing together got harder.
As implementation got cheaper, keeping a coherent whole mattered **more**. Left to themselves the agents converged on the ordinary layout, the common grid, the default colormap. They reach for whatever already exists somewhere in the training data.
- 5

Three parallel sessions was my limit.
Running more of them sped up feature development, but only to a point. Past about **three at once** I could not keep track of who was doing what. I gave commands in the wrong terminal, built features on the wrong branch, and lost track of work often enough that some of it was abandoned, or rebuilt from scratch by another session.

- 3

Verification was the hardest part.
Interactive behavior was the worst of it, for a structural reason. The agents observed the browser through **still screenshots** and had to infer what happened in between. Features often looked correct and did not work. Fixing one broke another without warning.

Where the effort went

Schematic. Impressions from one project, not measurements.

WRITING IT MYSELF

Implementation

Spec

Integ.

Verify

DIRECTING AGENTS

Specification

Integration

Verification

Writing code

Saying what it must do

Making the parts fit

Proving it is right

OPEN QUESTIONS FOR THIS COMMUNITY

An agent can inspect what it wrote, but it cannot fully observe what it **does**. In scientific software, that gap is exactly where the errors hide.

I KNOWING IT WORKS

- When an AI produces prodigious amounts of code, how do we know it does what it should?
- Generation got easy and stayed easy. Checking never did. Is that asymmetry the ceiling?
- 181,000 lines. How do I know it is secure? And who catches code that runs, passes its tests, and is *physically* wrong? Flipped signs, unit slips, staggered-grid offsets, silent regridding, the wrong CRS.

II WHAT WE ARE ALL BUILDING

- With thousands of people building overlapping tools, how do we prevent endless duplication, and do we even need to?
- Will most software eventually be generated on demand for one problem, then thrown away?
- Agents write too much rather than too little. How do you deliver the most value in the least code?

III SINGLE POINT OF FAILURE

- One person can now generate an enormous codebase, which makes them a single point of failure. What happens to the project when they stop?
- If vision is the scarce resource, how do you hand it to a successor who never had it?
- If one person can supervise about three agents at once, is human attention the real limit on how fast a group moves, rather than compute or model quality?

IV PEOPLE AND TOOLS

- If agents do the work, how do we train new scientists, and where does the judgment to review the code come from?
- A more capable assistant is not automatically a better one to work with. When a new model ships, do you switch, and how would you know?
- If an agent writes the model, the training loop, *and* the evaluation, what independence is left?

As the cost of intelligence drops rapidly toward zero, the scarce resource turns out to be **artistic vision and judgment**.

If that is right:
are we selecting for it, teaching it,
or rewarding it?